SIMATS ENGINEERING

Scenario Based Assignment – Assessment Tool

Course: Artificial Intelligence **Course Code:** CSA17

CO2: Search and game-solving techniques (Greedy, A*, CSP, Minimax, Alpha-Beta)

Name: Virupksha P **Register No:** 192417007

Question 1: AI Drones for Emergency Medicine Delivery (Greedy vs A*)

Scenario: *Drones deliver medicines in a flood zone. Roads get blocked suddenly, weather changes travel cost, the drone has a straight-line distance heuristic and only partial real-time updates, and delay must be minimal.*

1.1 Behaviour of Greedy Best-First Search (GBFS)

GBFS expands the node with the lowest heuristic value only, i.e. **f(n) = h(n)**, where h(n) is the straight-line distance to the delivery point. The drone therefore always flies/moves towards the neighbouring waypoint that looks closest to the goal and ignores the cost already spent (g(n)) and the terrain/weather cost of the next step.

- **Strengths:** very fast decision making, small memory use, and quick reaction when new information arrives; useful when time is more important than perfection.
- **Weaknesses:** not optimal and not complete on graphs with loops unless a closed list is kept. It can walk into a flooded valley or a storm because the point is geometrically close to the goal. It can also get trapped in dead ends (e.g. a river between the drone and the goal), because the straight line ignores obstacles.
- **Under uncertainty:** GBFS trusts a heuristic that is wrong in floods (straight-line distance is not real distance), so a small error in blocked-path information can make it commit to a risky route with no cost awareness.

1.2 Behaviour of A*

A* uses **f(n) = g(n) + h(n)**, where g(n) is the actual accumulated cost from the start (distance + weather penalty + terrain risk) and h(n) is the estimated remaining cost. It balances the two: g(n) keeps the search honest about what has already been spent, and h(n) pulls the search toward the goal. A node that looks close but was reached through very costly terrain gets a high f value and is not expanded early.

- If h(n) is **admissible** (never overestimates the real cost) and consistent, A* is **complete and optimal**. Straight-line distance is admissible when cost per unit distance is at least 1.
- Weather can be encoded in g(n) (e.g. cost = distance × weather factor), so A* prefers a slightly longer but safer, cheaper path.
- Weakness: it stores many nodes, so it is slower and more memory-hungry than GBFS on a large map.

1.3 Impact of Heuristic Accuracy

Heuristic quality	Greedy Best-First	A*
Accurate (close to true cost)	Very fast, near-direct path, often good result	Expands very few nodes, optimal path
Underestimates a lot (h too low)	Behaves randomly, loses guidance	Remains optimal but expands many nodes (approaches Uniform Cost Search)
Overestimates (inadmissible)	May go wrong without noticing	Loses optimality; may skip the best route
h = 0	Blind search	Becomes Dijkstra / UCS

Conclusion: Greedy is *very sensitive* to heuristic errors since the heuristic is its only guide, while A* is more robust because g(n) corrects the error. In floods, straight-line distance underestimates real cost, so A* stays safe (optimal) but slower, whereas Greedy can become dangerous.

1.4 Effect of Dynamic Changes (Blocked Paths)

- When a path is blocked, the previously computed path becomes invalid. Greedy simply picks the next best h value, which is cheap, but may lead into another dead end.
- A* must update edge costs (blocked edge = infinite cost) and re-plan; a full re-search is expensive in real time.
- The better approach is an **incremental/replanning variant** such as D* Lite or Lifelong Planning A*, which reuses earlier search effort and only repairs the affected part of the path.
- Partial information means the drone should treat unknown edges optimistically at first and update as sensors report, using receding-horizon replanning every few seconds.

1.5 Recommendation

Recommended: A* (with a weather-aware cost function) and replanning, i.e. D* Lite / Anytime A*. Justification:

- **Optimality:** minimum total delay and risk, which is the real objective for medicine delivery.
- **Safety:** terrain/weather costs and blocked edges are part of g(n), so dangerous areas are avoided. Greedy ignores them.
- **Real-time:** use a weighted A* (f = g + w·h, w slightly above 1) or Anytime A* for a fast first route which is improved if time allows; replan incrementally on updates.
- Greedy may only be used as a fallback emergency mode when computation time is almost zero.

Question 2: Smart-City Traffic Signal Optimization

Scenario: *AI must tune signal timings across hundreds of intersections to minimize waiting time, fuel use and congestion. The search space is huge, traffic changes dynamically, and the system can get stuck in local optima.*

2.1 Optimization Formulation

- **State:** the set of signal timings for all intersections (green duration, cycle length, offsets between neighbouring signals).
- **Decision variables:** x = (g_1, g_2, ..., g_n, offsets) where each g_i lies between a minimum and maximum green time.
- **Objective function (minimize):** Cost(x) = w_1·TotalWaitingTime + w_2·FuelConsumption + w_3·CongestionIndex, where w are importance weights.
- **Constraints:** minimum green time for pedestrians, maximum cycle length, emergency vehicle priority, safety intervals (yellow/all-red).
- **Neighbour/move operator:** change the timing of one or more signals slightly (±2 s) or shift an offset.

Why traditional (uninformed/systematic) search is inefficient: if each of 300 intersections has even 20 possible timings, the number of states is 20^{300}, so BFS/DFS/A* cannot enumerate or store them. There is no single goal state and no path cost, since we want the best configuration, not a route. Traffic is stochastic, so a stored solution becomes outdated quickly. Local search methods use constant memory and improve a current solution, which fits this problem.

2.2 Hill Climbing and Its Limitations

Hill Climbing starts with a current timing plan, generates neighbouring plans, moves to the neighbour with the best (lowest) cost, and repeats until no neighbour is better.

```
current = initial_plan
loop:
    neighbour = best_neighbour(current)
    if cost(neighbour) >= cost(current): return current
    current = neighbour
```

- Pros: simple, fast, low memory, easy to run online.
- Cons: gets stuck at local optima, wanders on plateaus, struggles on ridges, depends on the starting point, and is not complete or optimal.

2.3 Local Maxima, Plateaus and Ridges (Real-World Meaning)

Problem	Meaning in the traffic system
Local maximum/minimum	A timing plan where every small change makes things worse, e.g. a junction tuned so well locally that changing it would hurt it, though a completely different coordinated plan would reduce city-wide delay.
Plateau	Many neighbouring timings give the same total waiting time (e.g. during low night traffic), so the algorithm has no direction and may wander or stop.
Ridge	The improvement needs two or more signals to change together (e.g. a 'green wave' along a main road needs coordinated offsets). Changing one signal at a time never shows improvement.

2.4 Overcoming the Limitations

Simulated Annealing (SA): accepts a worse move with probability $\mathbf{e^{-\Delta E/T}}$, where T is the 'temperature'. At high T it explores widely (accepting worse timings), and as T is lowered it becomes selective like hill climbing. This allows the system to escape local optima and converge to a good global solution. It is easy to implement, uses little memory and its cooling schedule can be reset when traffic patterns change.

Genetic Algorithm (GA): keeps a population of signal plans (chromosomes = list of timings), evaluates fitness using a traffic simulator, selects the best, then applies *crossover* (combine parts of two plans, which can solve ridge problems) and *mutation* (random change to keep diversity). It explores many regions in parallel and can find coordinated multi-signal patterns.

Feature	Hill Climbing	Simulated Annealing	Genetic Algorithm
Escapes local optima	No	Yes (probabilistic)	Yes (population diversity)
Memory	Very low	Very low	Higher (population)
Speed	Fastest	Fast	Slower (many evaluations)
Handles ridges	Poor	Moderate	Good (crossover)
Parallelizable	No	Limited	Highly

2.5 Recommendation

Recommended: a hybrid of Genetic Algorithm (global exploration, run offline/periodically on traffic-simulation data) with Simulated Annealing or Hill Climbing (fast online fine-tuning of the best plan).

- **Scalability:** GA evaluates candidates in parallel across city zones; SA/HC use constant memory. Large problems can be divided into zones with neighbouring coordination.
- **Adaptability:** as traffic changes, the population or SA temperature is re-heated and the current best plan is used as a warm start, so the system adapts without restarting.
- **Reliability:** randomness reduces the risk of local optima, while the hill-climbing step guarantees stable real-time improvement.

Question 3: Autonomous Mars Rover (Online Search)

Scenario: *A rover explores unknown terrain, must avoid craters and rocks, has limited sensors, has no complete map, and updates knowledge as it explores.*

3.1 Why Online Search Instead of Offline Search

- **Offline search** computes the entire plan before acting. It needs a full, known model of the environment (map, transitions, costs). The rover does not have this.
- **Online search** interleaves *planning and acting*: the agent takes an action, observes the result and then decides the next step. This is the only possible way when the environment is unknown.
- Communication delay between Earth and Mars (minutes) rules out remote step-by-step control, so the rover must decide autonomously.
- Exploration itself is the way to discover the map; in addition, mistakes are irreversible (a fallen rover cannot be reset), so decisions must use freshly sensed data.

3.2 Challenges: Partial Observability and Dynamic Environment

- **Limited sensing range:** the rover sees only nearby terrain, so it cannot know what is behind a hill.
- **Sensor noise/uncertainty:** a shadow may be mistaken for a crater; dust reduces camera accuracy.
- **Unknown state space:** the number of states is not known, so the algorithm cannot be complete in advance and exploration could be endless.
- **Irreversible actions (unsafe explorability):** a dead end or steep slope may not allow the rover to return.
- **Dynamic changes:** dust storms, moving sand, rock falls and changing light conditions alter terrain information.
- **Resource limits:** battery, time and computation limit how much it can explore or re-plan.

3.3 Building and Updating the Internal Model

The rover maintains a **belief/world model** made of an occupancy grid or graph map, where each cell stores free / obstacle / unknown plus a probability. The cycle is:

- **Sense:** cameras, LIDAR and IMU give local observations.
- **Localize:** estimate its own position (SLAM: Simultaneous Localization and Mapping, with Kalman/particle filter).
- **Update the map:** mark cells as hazard/safe, update cost estimates and store visited states (e.g. result[s, a] = s' in the online agent's table).
- **Plan:** run a heuristic search (e.g. LRTA* or D* Lite) over the current map to select the next action.
- **Act and repeat:** execute one step, then sense again and correct the model if prediction and observation differ.

In LRTA* (Learning Real-Time A*) the agent keeps a table H(s) of cost estimates and after each move updates H(s) = min over actions of [cost(s,a,s') + H(s')], so estimates become more accurate as it learns the terrain.

3.4 Online Search vs Uninformed and Informed Search

Aspect	Uninformed (BFS/DFS/UCS)	Informed (Greedy/A*)	Online (LRTA*, Online DFS, D* Lite)
Knowledge needed	Full map	Full map + heuristic	Only local observations
Planning	Offline	Offline	Interleaved plan–act
Handles unknown/changing terrain	No	No (re-planning is costly)	Yes

Aspect	Uninformed (BFS/DFS/UCS)	Informed (Greedy/A*)	Online (LRTA*, Online DFS, D* Lite)
Optimality	BFS/UCS yes	A* yes (admissible h)	Not guaranteed, improves with learning
Memory	High	High	Low–moderate (local model/H table)
Safety on Mars	Unsafe for real use	Good if map known	Best, uses current sensing and replanning

3.5 Most Suitable Approach

Recommended: an online informed search with continuous replanning, e.g. D* Lite or LRTA*, on top of an incrementally built map (SLAM), with a safety layer.

- **Uncertainty:** a probabilistic map and sensor filtering represent doubt; cells with uncertain data get higher cost.
- **Adaptability:** D* Lite repairs only the affected portion of the plan when new obstacles are seen, so it is efficient.
- **Safety:** add risk-aware costs (slope, roughness), conservative margins around hazards, a rule that only moves to states from which return is possible, and a safe-mode stop when confidence is low.
- **Goal-directedness:** the heuristic (distance to the sample site) guides exploration so energy is not wasted.

Question 4: University Exam Timetabling as a CSP

Scenario: *An AI system should schedule exams so that no student has overlapping exams, exam halls are limited, some subjects must precede others, faculty are available, and special accommodations are met, with growing numbers of courses and students.*

4.1 CSP Formulation: Variables, Domains, Constraints

A CSP is defined by a triple **(X, D, C)**: variables, their domains, and constraints.

- **Variables (X):** each exam E_i (one for every course), optionally together with a hall variable H_i and invigilator variable F_i. Example: E_{AI}, E_{DBMS}, E_{Maths}.
- **Domains (D):** the possible (day, time-slot) pairs for each exam, e.g. {Day1-Morning, Day1-Afternoon, ..., Day10-Afternoon}; the hall domain is the set of available rooms. Domains are reduced up front for special cases (e.g. faculty unavailable on Day 3, so that slot is removed).

Constraint	Type	Formal expression / explanation
No student overlap	Binary (hard)	If courses Ci and Cj share at least one student, then slot(Ci) ≠ slot(Cj).
Limited halls	Global / capacity (hard)	For every slot t, number of exams with slot = t (or total students) must not exceed number of halls (or total seat capacity).
Precedence	Binary (hard)	If subject A must be before B then slot(A) < slot(B) (e.g. Maths-1 before Maths-2).
Faculty availability	Unary (hard)	slot(Ci) ∈ AvailableSlots(faculty of Ci); one invigilator cannot be at two exams in the same slot.
Special accommodation	Unary / binary (hard)	Students needing extra time/special rooms get a hall with facility and a slot with no consecutive exam; hall(Ci) ∈ AccessibleHalls.
Preferences (optional)	Soft	Avoid two exams for a student on the same day; spread exams evenly. Treated as costs to minimize.

The constraint graph has an edge between two exams if they share students, faculty or must be ordered. Timetabling is NP-complete, since it is equivalent to graph colouring (colour = time-slot).

4.2 Backtracking Search

Backtracking is depth-first search that assigns one variable at a time and checks consistency with the constraints already satisfied, and backs up when a violation occurs.

```
BACKTRACK(assignment):
  if all exams assigned: return assignment
  e = SELECT_UNASSIGNED_VARIABLE()
  for slot in ORDER_DOMAIN_VALUES(e):
     if consistent(e, slot, assignment):
         assignment[e] = slot
         result = BACKTRACK(assignment)
         if result != failure: return result
         remove assignment[e]
  return failure
```

Example: assign E_{AI} = Day1-Morning. Then E_{DBMS} (sharing students) cannot use that slot, so it tries Day1-Afternoon, and so on. If a later exam has no legal slot, the algorithm backtracks to change an earlier assignment. Plain backtracking may still waste time exploring hopeless branches, so heuristics are used: **MRV** (choose the exam with the fewest legal slots), **Degree heuristic** (choose the exam with most conflicts) and **LCV** (try the slot that rules out the fewest options for neighbours).

4.3 Constraint Propagation (Forward Checking and Arc Consistency)

- **Forward checking:** whenever an exam is assigned a slot, remove that slot from the domains of the neighbouring (conflicting) exams. If any domain becomes empty, backtrack immediately, so

failures are detected early rather than deep in the tree.

- **Arc consistency (AC-3):** for every constraint arc (Xi, Xj), delete values from Xi that have no supporting value in Xj. It can find inconsistency before search begins (e.g. precedence chain exceeding available days).
- **MAC (Maintaining Arc Consistency)** runs AC-3 after each assignment, giving stronger pruning than forward checking.

Result: smaller search tree, far fewer backtracks, and faster discovery of infeasible partial schedules. Example: after fixing $E_{Maths\text{-}1}$ on Day 2, forward checking removes all Day 1–2 slots for Maths-2 immediately.

4.4 Real-World Challenges and Best Strategy for Scalability

- **Exponential growth:** thousands of courses give an enormous search space; some instances may have no feasible solution.
- **Conflicting hard/soft constraints:** need prioritization; unsatisfiable requirements require relaxation.
- **Last-minute changes:** faculty or hall changes need quick repair without rebuilding the whole timetable.
- **Fairness:** students with many exams, accommodations and equal gaps between exams.

Best strategy: a combination of (1) problem decomposition (by department/day); (2) preprocessing with AC-3; (3) backtracking with **MRV + Degree + LCV + forward checking/MAC** for the hard constraints; (4) **Min-Conflicts local search** (or Simulated Annealing/Tabu search) to repair conflicts when the timetable changes, since it scales well and is quick for large problems; and (5) handling soft constraints as an optimization objective after a feasible schedule is found. This gives completeness for small/medium instances and scalability with fast repair on large ones.

Question 5: Strategic Game AI (Minimax and Alpha-Beta Pruning)

Scenario: *An AI opponent must compete against humans in a real-time strategy game, make decisions within strict time limits, evaluate complex game states, and handle an extremely large decision tree.*

5.1 How Minimax Models the Problem

Minimax treats the game as a **two-player, zero-sum, adversarial search**. The game tree contains states as nodes and moves as edges. **MAX** is the AI (tries to maximize its score) and **MIN** is the human (assumed to play optimally to minimize the AI's score). Leaf (or depth-limit) states have utility values. Values propagate upward: a MAX node takes the maximum of its children and a MIN node takes the minimum.

```
MINIMAX(s, depth):
  if terminal(s) or depth == 0: return EVAL(s)
  if player(s) == MAX: return max(MINIMAX(child, depth-1))
  else:                return min(MINIMAX(child, depth-1))
```

The AI selects the move leading to the child with the best minimax value, which guarantees the best outcome against a perfect opponent. In RTS games the state is the units, resources, positions and buildings, and the actions are move, build, attack, etc.

5.2 Computational Challenges of Large Game Trees

- **Exponential complexity:** time $O(b^d)$ with branching factor b and depth d. In an RTS, b can be hundreds (many units × many actions), so even depth 5–6 is impractical.
- **Time limits:** a decision must be made within milliseconds to seconds, so a full search is impossible.
- **Imperfect information and real time:** fog of war, simultaneous moves and continuous time make exact modelling difficult.
- **Evaluation cost:** evaluating every state is expensive; repeated states (transpositions) waste effort.
- **Memory:** the tree cannot be stored; depth-first approach uses O(b·d).

5.3 How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta is Minimax plus pruning, and it returns **exactly the same result** as Minimax. It maintains two values: **α** = best value (highest) that MAX can already guarantee along the path, and **β** = best value (lowest) MIN can already guarantee. When **α ≥ β**, the remaining children of that node cannot influence the final decision, so they are pruned.

Reasoning with an example: MAX has a first move with value 6. For the second move, MIN examines its first reply and finds value 3. Since MIN will choose ≤ 3 and MAX can already get 6 by the first move, MAX will never choose the second move, so MIN's other replies need not be examined (cut-off).

- **Worst case:** no pruning (bad move ordering), $O(b^d)$.
- **Best case (perfect ordering):** $O(b^{d/2})$; the search can go **twice as deep** in the same time or examine the square root of the nodes.
- **Average:** about $O(b^{3d/4})$ with random ordering.
- **Move ordering** is crucial: try best moves first using captures/attacks first, killer moves, history heuristic, and the best move from the previous iteration (iterative deepening).

5.4 Evaluation Function Design

As the tree cannot be searched to the end, the search is cut at a depth limit and a heuristic **EVAL(s)** estimates the value of the state. A weighted linear function is typical:

$EVAL(s) = w_1 \cdot f_1(s) + w_2 \cdot f_2(s) + \ldots + w_n \cdot f_n(s)$

Feature	Factor considered	Justification
Material / army strength	Count and value of own units minus enemy units	Units determine the ability to win fights
Resources and economy	Gold, wood, income rate, worker count	Economy decides long-term power
Map/territory control	Area controlled, key points, vision	Control gives resources and safe movement
Position / formation	Unit positions, chokepoints, high ground, distance to base	Positional advantage improves combat results
Technology and production	Upgrades, buildings, production capacity	Future strength potential
Threat and safety	Base health, enemy units near base, vulnerable units	Avoid fatal losses (king-safety equivalent)
Mobility / options	Number of legal useful moves	More choices = flexibility

The weights can be set by expert knowledge or tuned by self-play / machine learning. The function must be **fast to compute**, give higher values for better states for MAX, and be consistent with true outcomes at terminal states (win = $+\infty$, loss = $-\infty$).

5.5 Strategy to Balance Optimality and Real-Time Performance

- **Iterative deepening with time limit:** search depth 1, 2, 3... and return the best move found when time runs out (anytime behaviour).
- **Alpha-Beta with good move ordering** and a **transposition table** to avoid re-evaluating repeated states.
- **Depth-limited search + evaluation function** (and quiescence search to avoid evaluating in the middle of a battle).
- **Abstraction and selective search:** consider only the most promising moves (beam / forward pruning), group units into squads and use high-level actions (script-based moves).
- **Hybrid techniques:** Monte Carlo Tree Search (MCTS) for very large or uncertain games, and opening books/precomputed strategies for early game.
- **Parallel computation** on multiple cores and caching of results between frames.

Recommendation: use **iterative-deepening Alpha-Beta search with a tuned evaluation function, move ordering and transposition tables**, and fall back on the best move found so far when the time limit is reached. This gives near-optimal decisions with guaranteed response time, and can be combined with MCTS in the very large strategic layer.

Summary of Key Recommendations

Q	Problem	Recommended technique
1	Drone route planning	A* with weather-aware costs + replanning (D* Lite / Anytime A*)
2	Traffic signal optimization	Hybrid GA (global) + Simulated Annealing/Hill Climbing (fine-tuning)
3	Mars rover navigation	Online informed search (D* Lite / LRTA*) with SLAM map and safety layer
4	Exam timetabling	CSP: Backtracking + MRV/Degree/LCV + forward checking/MAC; Min-Conflicts for repair
5	RTS game AI	Iterative-deepening Alpha-Beta with evaluation function and move ordering (+ MCTS)